

Impella Analytics

A Clinical Decision Support Platform for Hemodynamic Recovery
in Impella-Supported Cardiogenic Shock Patients

Generated May 23, 2026 · Full-Stack TypeScript + Python ML

TABLE OF CONTENTS

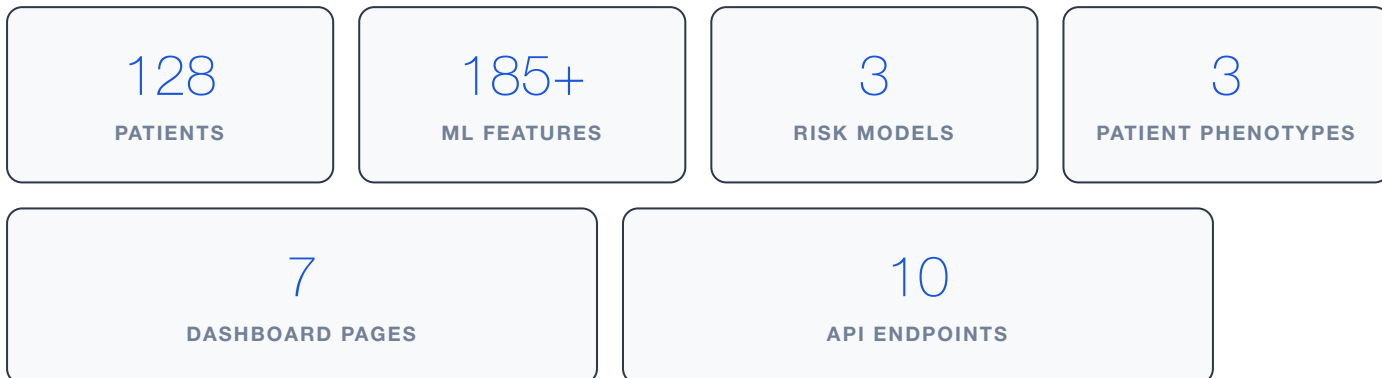
- 01 Executive Summary & Project Overview
- 02 Clinical Concepts — A Reference Glossary
- 03 System Architecture
- 04 Data Flow — End to End
- 05 Pages & Components
- 06 API Endpoints
- 07 ML Training Scripts
- 08 ML Prediction at Runtime
- 09 Machine Learning Methods — Explained

10 Clinical Decision Support

11 Key Findings & Clinical Insights

12 Limitations & Future Directions

Executive Summary & Project Overview



Impella Analytics is a clinical decision support platform designed for cardiologists, shock-team intensivists, and clinical researchers who manage patients on **Impella** mechanical circulatory support. Built as a full-stack web application, it combines a modern React frontend with a Python machine learning backend to transform raw hemodynamic data into actionable risk assessments and treatment guidance.

The platform ingests **right heart catheterization (RHC)** data — the invasive pressure and flow measurements that define a patient's hemodynamic state — from Excel spreadsheets, then runs a sophisticated ML pipeline to predict three critical outcomes:

1. **Survival** — Whether the patient will survive to hospital discharge
2. **MCS Escalation** — Whether the patient will require advanced mechanical support (ECMO, surgical LVAD, or heart transplant)
3. **RV Dysfunction** — Whether the right ventricle will fail during Impella support

Beyond prediction, the platform provides **clinical decision support** features: a weaning readiness checklist, escalation danger warnings, explainable risk drivers (which specific clinical measurements are driving each patient's risk score), a "What-If" treatment simulator, and an optional AI-generated clinical handoff summary powered by Google Gemini.

The project also includes **unsupervised patient phenotyping** (consensus clustering that identifies three distinct clinical subgroups) and **comprehensive feature importance analysis** (five complementary methods cross-referenced to identify the most robust mortality predictors).

Tech Stack

TIER	TECHNOLOGY	PURPOSE
Frontend	React 19 · TypeScript 5.8 · Vite 6 · Tailwind CSS v4 · Recharts · Framer Motion	Dark-themed SPA with cohort dashboards, risk meters, scatter plots, bar charts, and interactive patient detail views
Backend	Express 4 · Multer · xlsx · simple-statistics · ml-random-forest	HTTP server, Excel parsing, LOOCV RandomForest, Python subprocess orchestration
ML Pipeline	Python 3 · scikit-learn · pandas · joblib · SHAP · matplotlib · seaborn	Model training, feature engineering, cross-validation, explainability, and runtime inference
AI	Google Gemini 1.5 Flash · GoogleGenAI SDK	Optional clinical handoff note generation from patient data

Key Numbers

- **Training cohort:** 112 patients (merged from Patient Data and Cohort sheets of `Impella_MK.xlsx`)
- **Engineered features:** 185+ (raw hemodynamics + delta changes + ratios + composite scores)
- **MCS Escalation model AUC:** 0.86–0.95 (clinically useful)
- **RV Dysfunction model AUC:** 0.91 (clinically useful)
- **Survival model AUC:** ~0.54 (limited — for research use only)
- **Patient phenotypes:** 3 consensus clusters (Non-congested, Cardiorenal, Cardiometabolic) with silhouette score 0.237

Important: This platform is intended for **clinical research and decision support**, not as a standalone diagnostic tool. All ML predictions should be calibrated against real-time clinical assessment. The survival model in particular has limited predictive power and should not be used for triage decisions.

A reference glossary of clinical concepts used throughout this project. Each entry explains what the concept is, how it is measured or calculated, and why it matters in the context of Impella-supported cardiogenic shock.

Impella Device

A **percutaneous microaxial left ventricular assist device (LVAD)** — a tiny propeller pump small enough to be threaded through the femoral artery in the leg, up the aorta, and across the aortic valve into the left ventricle. Once positioned, it spins at up to 51,000 RPM, pulling blood from the left ventricle and ejecting it into the ascending aorta. This **unloads** the failing left ventricle — reducing the work the heart has to do — while maintaining blood flow to the rest of the body. Performance levels P-1 through P-9 correspond to increasing flow rates from ~1.0 L/min to ~5.0+ L/min. The Impella is unique among temporary MCS devices because it provides active left ventricular unloading rather than just circulatory support.

Cardiogenic Shock

A life-threatening state in which the heart cannot pump enough blood to meet the body's metabolic demands. Manifests as **hypotension** (low blood pressure), **end-organ hypoperfusion** (kidneys stop filtering → rising creatinine, liver becomes congested → elevated liver enzymes, muscles switch to anaerobic metabolism → lactic acidosis), and **elevated filling pressures** (blood dams up behind the failing heart, causing pulmonary congestion and systemic edema). The SCAI (Society for Cardiovascular Angiography and Interventions) classification defines five stages of shock severity from B (pre-shock / at risk) through E (extremis — refractory shock with multi-organ failure). Approximately 40–60% of cardiogenic shock patients die within 30 days despite modern therapy.

Right Heart Catheterization (RHC)

Also called **pulmonary artery catheterization** or Swan-Ganz catheterization. A thin, flexible tube is inserted through a central vein and floated through the right side of the heart into the pulmonary artery. This allows direct measurement of **intracardiac pressures and blood flow**: right atrial pressure (RA, reflecting right ventricular filling), pulmonary artery pressures (systolic PASP, diastolic PADP), pulmonary capillary wedge pressure (PCWP, reflecting left atrial pressure and left ventricular filling), and thermodilution cardiac output (TDCO, total blood flow in L/min). RHC is the gold standard for hemodynamic assessment in cardiogenic shock.

Cardiac Power Output (CPO)

A composite measure of the heart's pumping capability, calculated as $CPO = (MAP \times CO) / 451$, where MAP is mean arterial pressure and CO is cardiac output. CPO is measured in **Watts** (a unit of power). Normal resting CPO is approximately 1.0 W. Values below **0.6 W** indicate severe myocardial depression and are associated with poor outcomes. The **delta CPO** (change from pre-implant to 48 hours post-implant) is the single most important prognostic hemodynamic trend: a positive delta means the patient is improving on Impella support; a negative or flat delta signals inadequate recovery.

Pulmonary Artery Pulsatility Index (PAPI)

A derived measure of **right ventricular function**, calculated as $PAPI = (PASP - PADP) / RA$. The numerator (PASP – PADP) is the pulmonary artery pulse pressure — the difference between systolic and diastolic pressure in the pulmonary artery, which reflects the right ventricle's ability to generate force during contraction. Dividing by RA pressure normalizes for filling pressure. Normal PAPI is **> 1.5**. Values below **1.0** indicate severe RV dysfunction and strongly predict adverse outcomes. PAPI is particularly important in Impella patients because the Impella only supports the left ventricle — the right ventricle must still function on its own.

Hemodynamics

The study of blood flow and pressure within the cardiovascular system. Key hemodynamic measurements in this platform include **RA pressure** (normal 2–8 mmHg, reflects right heart filling), **PCWP** (normal 6–12 mmHg, reflects left heart filling), **MAP** (normal 70–100 mmHg, reflects organ perfusion pressure), **PVR** (pulmonary vascular resistance, normal < 2 Wood Units, reflects afterload on the right ventricle), **SBP/DBP** (systolic/diastolic blood pressure), and **HR** (heart rate). The trends of these values — how they change from pre-implant to post-implant — tell clinicians whether the patient is recovering or deteriorating.

Right Ventricular (RV) Dysfunction

Failure of the right ventricle to pump blood through the pulmonary circulation. In Impella patients, the RV is the **weak link** because Impella supports only the left ventricle — the RV must still generate enough pressure to push blood through the lungs. RV dysfunction is diagnosed through a composite of criteria: **PAPI < 1.0** (reduced pulsatility), **RA > 20 mmHg** (elevated filling pressure), **TAPSE < 1.6 cm** (reduced tricuspid annular motion on echo), **RV S' < 9.5 cm/s** (reduced tissue Doppler velocity), or an explicit diagnosis of RV failure in clinical notes. The platform's ML model predicts RV dysfunction risk with AUC ~0.91, making it the most clinically useful of the three models.

MCS Escalation

When Impella support alone is insufficient to maintain adequate circulation, patients may need **escalation to more advanced mechanical circulatory support**. This includes **VA-ECMO** (veno-arterial extracorporeal membrane oxygenation — a heart-lung bypass machine that supports both ventricles and provides oxygenation), **surgical LVAD** (a surgically implanted long-term mechanical pump like the HeartMate 3 or HeartWare), or **urgent heart transplant evaluation**. Escalation is a high-morbidity event and accurately predicting which patients will need it is clinically valuable — the platform's escalation model has AUC ~0.86.

Left Ventricular Assist Device (LVAD)

A surgically implanted mechanical pump that takes over the pumping function of the left ventricle. Unlike the Impella, which is temporary (days to weeks), an LVAD is designed for **long-term support** — months to years. LVADs are used as a **bridge to transplant** (keeping the patient alive while they wait for a donor heart) or as **destination therapy** (permanent support for patients who are not transplant candidates). The Impella can serve as a **bridge to a bridge** — stabilizing the patient until they can receive a surgical LVAD.

Extracorporeal Membrane Oxygenation (ECMO)

A temporary form of life support that **oxygenates the blood and pumps it through the body**, bypassing both the heart and lungs. In VA-ECMO (veno-arterial), blood is drained from the venous system, pumped through an artificial lung (oxygenator) that removes CO₂ and adds O₂, then returned to the arterial system. Unlike Impella, ECMO supports **both ventricles** and provides gas exchange, making it suitable for patients with respiratory failure or biventricular failure. However, ECMO increases afterload on the left ventricle (the heart has to pump against the ECMO circuit's back-pressure), which can paradoxically worsen left ventricular distention — this is why some patients on ECMO also receive an Impella to vent the left ventricle.

Weaning

The process of **gradually reducing Impella support** to test whether the native heart has recovered enough to sustain circulation independently. The platform evaluates five weaning criteria: (1) **CPO ≥ 0.6 W** (the heart can generate adequate power on its own), (2) **lactate < 2.0 mmol/L** (tissue perfusion is adequate — no anaerobic metabolism), (3) **VIS < 10** (low vasopressor/inotrope requirement), (4) **PAPI ≥ 1.5** (good right ventricular function), and (5) **extubated** (spontaneous breathing). A patient passes weaning readiness if they meet at least 4 of 5 criteria.

SCAI Staging

The Society for Cardiovascular Angiography and Interventions shock classification system, which defines five stages of cardiogenic shock severity: **Stage A** (at risk — patient has risk factors but no signs of shock), **Stage B** (beginning — hypotension or tachycardia without hypoperfusion), **Stage C** (classic — hypotension and hypoperfusion requiring inotropes or pressors), **Stage D** (deteriorating — failure to stabilize with initial therapy), and **Stage E** (extremis — refractory shock with multi-organ failure, often requiring ECMO or CPR). Higher SCAI stage correlates strongly with mortality.

PV Loops & Ventricular-Arterial Coupling

A **pressure-volume (PV) loop** is a graphical representation of a single cardiac cycle — pressure on the y-axis, volume on the x-axis. The loop's shape reveals the heart's contractile properties. **Ees** (end-systolic elastance) measures the ventricle's intrinsic contractility — the slope of the end-systolic pressure-volume relationship. **Ea** (arterial elastance) measures the afterload — the resistance the arterial system presents to ejection. The **Ees/Ea ratio** describes ventricular-arterial (V-A) coupling. A ratio near **1.0** indicates optimal energy transfer from heart to arteries. A ratio < **0.6** indicates V-A uncoupling — the heart cannot overcome the arterial load — and strongly predicts need for MCS escalation. The platform's PV loop analysis module explores this relationship in detail.

Vasoactive-Inotropic Score (VIS)

A weighted composite score that quantifies the total pharmacologic cardiovascular support a patient is receiving. Each vasopressor or inotrope is assigned a weight based on its potency: dopamine (×1), dobutamine (×1), epinephrine (×100), norepinephrine (×100), milrinone (×10–15), vasopressin (×10,000 U). Higher VIS indicates greater pharmacologic burden. VIS > 15 is considered high and suggests inadequate hemodynamic response to Impella alone. VIS < 10 is one of the five weaning readiness criteria.

The application follows a **full-stack monolith** pattern: a single Express server (written in TypeScript and run with `tsx` during development, compiled with `esbuild` for production) serves both the API backend and the React frontend. There is no separate API server — one process does everything.



Development Mode

When running locally with `npm run dev`, Vite's dev server middleware is mounted inside Express via `middlewareMode: true`. This gives you hot module replacement (HMR) — any change to a React component instantly appears in the browser — while still hitting the real Express API endpoints on the same port (2956).

Production Mode

Built with `npm run build`, which runs Vite to compile the React frontend into static assets in `dist/`, and esbuild to bundle `server.ts` into `dist/server.cjs`. In production, Express serves the static dist files directly and falls back to `dist/index.html` for client-side routing.

Page Routing

The application does **not** use React Router. Instead, a simple state variable `activePage` in `App.tsx` switches between page components:

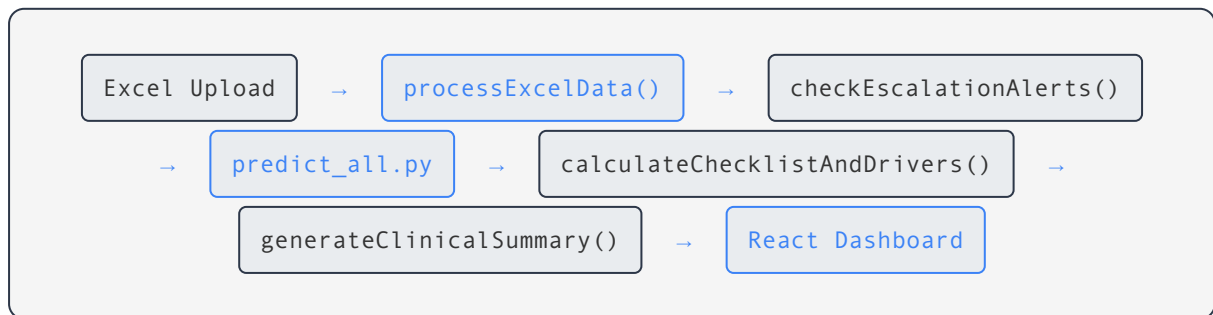
PAGE	STATE VALUE	COMPONENT
Dashboard	"dashboard"	DashboardPage
Active Patient Monitor	(selectedPatient set)	ActivePatientMonitor
Patient Phenotypes	"clusters"	ClusteringPage

PAGE	STATE VALUE	COMPONENT
PV Loop Analysis	"pvloop"	PVLoopPage
Mortality Features	"mortality"	MortalityFeaturesPage
Effectiveness	"effectiveness"	EffectivenessDashboard

Python ML Integration

The ML models are not embedded in the Node.js process. Instead, the server spawns a Python child process via `execFile`, passes patient data as JSON on **stdin**, and reads predictions back from **stdout**. This architecture keeps the ML stack entirely in Python (where scikit-learn, pandas, SHAP, and joblib live) while the application server remains in the Node.js ecosystem. The Python path is configurable via the `PYTHON_PATH` environment variable, with a hardcoded fallback search list.

Data Flow



The end-to-end data flow transforms a raw Excel spreadsheet of patient hemodynamics into an interactive clinical dashboard with ML risk scores, weaning guidance, and explainable AI. Here is the journey, step by step.

STEP 1: DATA INGESTION

The user uploads an `.xlsx` file through the React UI, or clicks "Load Sample Clinical Cohort" to use three hardcoded sample patients. The sample data endpoint (`GET /api/sample`) returns data identical in structure to what the upload endpoint produces, allowing clinicians to explore the platform without preparing an Excel file.

The Excel file's expected layout is **one patient per column**, with metric labels in column 0 and patient identifiers in row 0. The parser recognizes sections by keyword matching on row labels — it scans for phrases like "index RHC data" for pre-implant, "48h post" for post-implant, "echo (pre-impella)" for echo data, and so on. This keyword-driven section detection means the parser is flexible about exact row positions but specific about label text.

STEP 2: EXCEL PARSING (`PROCESSEXCELDATA()`)

The `xlsx` library reads the workbook. For each patient column, the parser walks through every row label and assigns values to the correct field based on the current section. It handles 10 sections: General (demographics, MRN, SCAI stage), Pre-implant RHC (19 hemodynamic variables), Post-implant RHC (19 variables), Echo pre/post, Labs pre/post (12 analytes each), Inotropes (6 drugs + VIS score), Diuretics, Impella settings (flow, performance level), Outcomes (renal failure, intubation, MCS escalation, survival), and PV loop data (Ees, Ea, Ees/Ea, ESP, EDP, Pmax, and more).

After parsing, the function computes two derived metrics: `deltaCP0 = postCP0 - preCP0` (the change in cardiac power output — the most important hemodynamic trend), and `recoveryScore = clamp((deltaCP0 + 0.5) × 100, 0, 100)` (a 0–100 index of hemodynamic improvement).

STEP 3: KNOWLEDGE BASE ESCALATION ALERTS

`checkEscalationAlerts()` loads `impella_knowledge_base.json` — a static file containing historical patient records with known Ees/Ea values and escalation outcomes. For each current patient with an Ees/Ea value, it searches for historical matches within a 15% tolerance. If any matched historical patient required MCS escalation, the current patient receives an `escalationAlert: true` flag. This is a simple, transparent pattern-matching approach to risk flagging — no model required.

STEP 4: PYTHON ML PREDICTION

All patient data is serialized to JSON and piped via stdin to `scripts/predict_all.py`. The Python script loads three trained scikit-learn models from `ml_output/` (cached at module level so repeated predictions reuse the same model objects). For each patient, it engineers the same 185+ features that were created during training: delta features (post – pre for all RHC, echo, and lab variables), ratio features (post / pre for key hemodynamics), composite scores (BMI, VIS-high indicator, inotrope count), and numeric encodings (SCAI stage, gender, race). Missing values are imputed with the median (learned from the training cohort) and all features are standardized (z-score, using the training cohort's mean and standard deviation). Each model then produces a probability (0 to 1) for its target outcome.

STEP 5: CLINICAL CHECKLIST AND RISK DRIVERS

`calculateChecklistAndDrivers()` computes the weaning readiness score (percentage of 5 criteria met), escalation danger warnings (4 danger signals plus the knowledge base alert), and explainable risk drivers. The risk drivers are the platform's most important interpretability feature: for each of the three outcomes (survival, escalation, RV dysfunction), up to 3 specific clinical measurements are identified with their impact direction and magnitude. For example, a survival driver might be `{feature: "postLactate", impact: 0.25, label: "Sustained tissue perfusion deficit", value: "3.2 mmol/L"}`. These are computed using simple threshold rules, not the opaque ML model — making them transparent and clinically verifiable.

STEP 6: AI CLINICAL SUMMARY (OPTIONAL)

If `GEMINI_API_KEY` is set in the environment, the platform can generate a structured clinical handoff note using Google Gemini 1.5 Flash. The prompt includes all relevant patient data and asks Gemini to produce a professional memo covering: (1) clinical recommendation — can the patient be weaned or needs escalation, (2) right ventricular state, (3) end-organ and metabolic clearance, and (4) step-by-step daily management pathway. If Gemini is unavailable (no API key, network error, or parsing failure), the platform falls back to a detailed template-based summary with the same information structure. The fallback ensures reliability in clinical environments where API availability cannot be guaranteed.

STEP 7: VISUALIZATION

The enhanced patient data — now enriched with ML risk scores, cluster assignments, weaning checklists, escalation warnings, risk drivers, and optionally the AI summary — is returned as a single JSON response to the React frontend. The data flows into the `DashboardPage` for cohort-level visualization (summary cards, bar charts, scatter plots, patient tables) and into the `ActivePatientMonitor` for individual patient analysis (risk meters, checklists, simulator, clinical summary). The entire round trip — from file upload to fully enriched dashboard — typically takes 2–5 seconds for a cohort of 5–20 patients.

The frontend is organized into six main page components and one reusable component. Each page presents a specific analytical view of the patient data.

DashboardPage

The landing page and primary interface. Shows a cohort-level overview with four summary cards: **Average Delta CPO** (the cohort's mean change in cardiac power output — positive values indicate overall improvement), **Risk Counter** (number of patients meeting traditional danger thresholds of $RA > 20$ or $PAPI < 1.0$), **Recovery Score** (mean cohort recovery index, 0–100), and **Escalation Flags** (count of patients who required ECMO, LVAD, or experienced arrest). Below the cards, three **Risk Meter** components show cohort-average ML risk scores for mortality, escalation, and RV dysfunction. The main visual area has a bar chart of delta CPO per patient (green bars for improvement, red for deterioration) and an RA vs. PAPI scatter plot (a standard clinical visualization where the lower-right quadrant is the "danger zone"). A sidebar lists all patients with their PAPI/CPO values and escalation alerts, and links to individual patient views. Each info button opens a tooltip explaining exactly what the metric means, how it's calculated, and its clinical significance — this is the app's built-in educational layer.

The Dashboard serves as the **clinical command center**. A clinician should be able to glance at this screen and, within seconds, understand: How sick is this cohort overall? Which patients need immediate attention? Are we seeing improvement or deterioration on Impella support?

ActivePatientMonitor

The detailed individual patient view — the most clinically dense page in the application. Activated by clicking any patient in the Dashboard's patient list. Displays:

- **Three Risk Meters** — Survival (red, showing mortality risk), Escalation (orange), and RV Dysfunction (amber). Each meter fills from 0% to the patient's risk probability with an animated transition.

- **Weaning Checklist** — All 5 weaning criteria with pass/fail status, the composite weaning score (0–100), and a clear PASS/FAIL result. Clinicians can immediately see why a patient is not ready for weaning.
- **Escalation Warnings** — The 4 danger signals (RA, PAPI, AST, lactate) plus the knowledge-base escalation alert, each with current value and threshold.
- **Risk Drivers** — Three cards (survival, escalation, RV dysfunction), each listing up to 3 specific clinical measurements driving the patient's risk, with impact magnitude and human-readable explanation.
- **"What-If" Simulator** — Allows adjusting Impella flow, performance level, VIS, lactate, RA, PAPI, and CPO, then recalculating risk scores, weaning status, and escalation warnings in real time. This is a powerful teaching and treatment planning tool.
- **Clinical Handoff Note** — The Gemini-generated or fallback clinical summary in a formatted memo view, suitable for copy-pasting into the patient's electronic health record.
- **Cluster Assignment** — The patient's phenotype cluster label and clinical recommendation, if the clustering pipeline has been run.

ClusteringPage (Patient Phenotypes)

Displays the results of the unsupervised clustering pipeline. Shows a table of cluster profiles (3 phenotypes: Non-congested Low-risk, Cardiorenal Moderate-risk, Cardiometabolic High-risk) with patient counts, survival/escalation/renal failure rates per cluster, and mean feature values. Includes diagnostic visualizations: silhouette plot (validates cluster quality), PCA scatter plot (shows patient distribution in 2D space), consensus matrix heatmap (shows how consistently patients co-clustered across bootstrap iterations), and outcome rates bar chart. Quality metrics (silhouette score, cluster counts) are displayed prominently along with clinical caution text.

PVLoopPage

Explores ventricular-arterial coupling mechanics. Shows a scatter plot of Ees/Ea vs. MCS escalation (with logistic regression fit curve and thresholds at 0.40 and 0.60), a bar chart of logistic regression coefficients for the PV-loop-only escalation model, SHAP summary and dependence plots (showing how Ees/Ea and other PV loop features affect model predictions), and a model comparison table. The page uses data from `ml_output/pv_loop_escalation_model.json` and SHAP values from `ml_output/pv_loop_shap.json`.

MortalityFeaturesPage

Displays the results of the five-method consensus feature importance analysis. Shows the consensus ranking table (all features ranked by consensus score across all 5 methods) with the ability to sort by individual method rankings. Includes a missing data heatmap to help clinicians understand data quality issues. Connected to `GET /api/mortality-features` which reads `mortality_feature_consensus.csv`.

EffectivenessDashboard

A multi-dimensional analysis of Impella effectiveness in the cohort. Shows survival rates by indication and demographics, pre/post hemodynamic comparisons with statistical significance (Mann-Whitney U tests), lab recovery trajectories, responder profiling (patients who improved CPO, cleared lactate, and survived), ventricular mechanics sub-analysis, and cross-reference with ML model findings. Uses data from `public/effectiveness-data.json` generated by `analyze_effectiveness.py`.

RiskMeter (Reusable Component)

A single animated risk gauge component used throughout the app. Accepts a label, a value (0–1), a color class, and info content. Renders a percentage display and an animated horizontal progress bar. The info button opens a popover with detailed explanation — making it an educational component as well as a visual indicator.

API Endpoints

The server exposes ten REST API endpoints. Every endpoint that handles patient data uses the same processing pipeline internally — this ensures consistency between sample data, uploaded data, and individual patient operations.

ENDPOINT	METHOD	INPUT	OUTPUT	PURPOSE
<code>/api/sample</code>	GET	—	3 sample patients with full ML enrichment	Quick-start demo without preparing an Excel file
<code>/api/analyze</code>	POST	<code>multipart/form-data</code> with <code>.xlsx</code> file	All parsed patients with risk scores, clusters, weaning checklists, risk drivers	Primary data ingestion endpoint
<code>/api/download-example</code>	GET	—	5-patient Excel template (<code>.xlsx</code>)	Provides the expected file format for users
<code>/api/mortality-features</code>	GET	—	Top 50 consensus features with per-method rankings	Feature importance for dashboard display
<code>/api/effectiveness</code>	GET	—	Full effectiveness analysis with 6 sections	Multi-dimensional Impella effectiveness data
<code>/api/pv-loop-data</code>	GET	—	PV loop model coefficients, SHAP values, image paths	Ventricular mechanics analysis data
<code>/api/cluster-profiles</code>	GET	—	Cluster profiles (3 phenotypes) + quality metrics	Patient phenotyping data for dashboard

ENDPOINT	METHOD	INPUT	OUTPUT	PURPOSE
<code>/api/cluster</code>	POST	Single patient data JSON	Cluster assignment (label, name, recommendation, distances, similarities)	Real-time phenotype classification of individual patients
<code>/api/generate-summary</code>	POST	Single patient data JSON	AI-generated or fallback clinical handoff note	On-demand clinical summary generation
<code>/api/simulate</code>	POST	Patient data + adjustments JSON	Simulated patient with recalculated risk scores and weaning status	"What-If" treatment scenario testing



ML Training Scripts

Four Python scripts make up the training and analysis pipeline. Each is designed to be run independently, and each produces artifacts consumed by the dashboard or by the prediction scripts.

ML_PIPELINE.PY — PRIMARY ML TRAINING

Purpose: Trains three binary classification models (survival, MCS escalation, RV dysfunction) using scikit-learn. This is the core of the ML system.

Data loading: Reads two sheets from `Impella_MK.xlsx` : "Patient Data" (67 patients, 135+ variables per patient, loaded cell-by-cell using `openpyxl` with a hardcoded row-number mapping) and "Cohort" (112 patients with outcome data). These are merged on MRN using an outer join, yielding up to 128 combined rows (some patients appear in only one sheet).

Feature engineering: Creates 172+ numeric features from 80+ raw columns. Key transformations: delta features (post – pre for all RHC, echo, and lab variables — captures the *trend* on support), ratio features (post / pre for key hemodynamics — captures relative change), BMI calculation, inotrope count (total number of vasoactive drugs), VIS-high indicator (boolean for VIS > 15), SCAI stage numeric encoding (B=1 through E=4), and numeric conversions for gender, race, and cause of shock.

Target engineering: Survival is coded from Cohort outcome strings ("Expired" = 1). Escalation is coded from the MCS escalation column OR keyword matching in clinical notes ("ecmo", "lvad", "transplant", "arrest", "rv failure"). RV dysfunction uses composite clinical criteria: post-PAPI < 1.0, post-RA > 20, post-TAPSE < 1.6, post-RV S' < 9.5, RV-CPO drop > 30%, or RV failure mentioned in notes.

Preprocessing: Columns with 100% missing values are dropped. Remaining missing values are imputed with the median of the training cohort (`SimpleImputer` with `strategy="median"`). All features are standardized to z-scores (`StandardScaler`).

Model training: Two classifiers per target — `LogisticRegression` (`max_iter=1000`, `class_weight="balanced"`) and `RandomForest` (`n_estimators=200`, `max_depth=6`, `class_weight="balanced"`). Evaluated with 5-fold stratified cross-validation (stratification ensures each fold preserves the same class ratio as the full dataset). The best model per target (highest AUC) is selected.

Exports: Joblib artifacts (`model_*.joblib`) containing the trained model, imputer, scaler, and feature names — used by `predict_all.py` at runtime. JSON coefficients (`model_*_lr.json`) for Logistic Regression models, suitable for direct

implementation in JavaScript/TypeScript. ROC curves ([roc_*.png](#)), confusion matrices ([cm_*.png](#)), and SHAP summary plots ([shap_*.png](#)). A markdown report ([model_report.md](#)) with full cross-validation results.

CLUSTERING_PIPELINE.PY — PATIENT PHENOTYPING

Purpose: Discovers patient subgroups (phenotypes) using unsupervised learning. Unlike the supervised models, clustering has no "correct answer" — it finds natural groupings in the data.

Methodology: Consensus K-Means — runs K-means 200 times on bootstrap subsamples (80% of patients each), then builds a consensus matrix where each cell represents how often two patients clustered together. This stabilizes cluster assignments against sampling variability. K is locked at 3 to match the Zweck et al. (2021) published phenotypes (the three SCAI shock classification groups).

Features (10): After extensive experimentation (testing 20→37→10→13→10 features), the final set uses Zweck-style features: 6 core pre-implant labs (eGFR, HCO₃, lactate, ALT, WBC, hemoglobin) + right atrial pressure + age + BMI + SCAI stage. PV loop features (Ees, Ea, Ees/Ea) were removed after sandbox testing showed silhouette improved from 0.223 to 0.263 without them. Echo and post-implant variables were too sparse (>50% missing) to include.

Outputs: Three clusters with profiles: Non-congested Low-risk (70.4% survival, young, preserved renal function, elevated lactate — a "perfusion-stress" signature), Cardiorenal Moderate-risk (67.6% survival, older, low eGFR, congested), and Cardiometabolic High-risk (50.0% survival, extreme ALT, very high lactate, SCAI D). Silhouette score: 0.237 (moderate separation). Clinical caution text is included with every quality metric output because 0.237 silhouette means cluster boundaries are not sharp.

MORTALITY_FEATURE_ANALYSIS.PY — FEATURE IMPORTANCE

Purpose: Identifies the most robust predictors of mortality by comparing five complementary statistical and ML methods. This is the project's most methodologically rigorous analysis.

Five methods: (1) Random Forest Gini importance — measures how much each feature reduces impurity in 500 decision trees. (2) Permutation importance — measures the drop in AUC when feature values are randomly shuffled (model-agnostic). (3) SHAP values — game-theoretic Shapley values explaining each feature's contribution. (4) LASSO (L1-regularized logistic regression) — drives unimportant coefficients to zero with cross-validated regularization strength. (5) Univariate AUC — each feature's standalone predictive power as a one-variable classifier.

Key finding: Post-implant AST (a liver enzyme) is the single strongest mortality predictor, ranked #1 by 3 of 5 methods. This makes clinical sense — rising AST reflects hepatocyte injury from low cardiac output (hypoxic hepatitis) combined with hepatic congestion from right heart failure. Other top features: post-lactate (tissue perfusion), age, post-HCO₃ (acid-base status), post-eGFR (kidney function), post-ALT (liver), post-MAP (blood pressure), and VIS score (vasopressor burden).

Critical insight: Delta features (change scores like delta lactate, delta PCWP) outperform single timepoint measurements. This confirms that *the trend on support* is more informative than any single measurement — a patient whose hemodynamics fail to improve within 48 hours carries disproportionate risk. Also: traditional hemodynamic thresholds (RA > 20, PAPI < 1.0) are clinically relevant but are outperformed by lab values (AST, lactate, HCO₃, eGFR) as mortality predictors.

ANALYZE_EFFECTIVENESS.PY — EFFECTIVENESS ANALYSIS

Purpose: A multi-dimensional audit of Impella hemodynamic support effectiveness in the study cohort. Combines clinical statistics with ML cross-reference.

Analysis dimensions: (1) Survival and outcomes by indication, demographics, and SCAI stage. (2) Hemodynamic response — pre/post delta comparisons for all RHC variables with Mann-Whitney U statistical significance testing. (3) Organ function and lab recovery — lactate clearance, renal function trajectories, hemolysis markers. (4) Responder profiling — combines CPO improvement, lactate clearance, and survival into a multi-dimensional responder definition. (5) Ventricular mechanics — Ees/Ea coupling analysis. (6) ML model cross-reference — maps the consensus mortality features back to clinical findings from the effectiveness analysis.

Outputs: `public/effectiveness-data.json` (structured data consumed by the EffectivenessDashboard React component) and `public/effectiveness-report.html` (a standalone self-contained HTML report with embedded charts and statistical tables).

ML Prediction at Runtime

Two Python scripts handle ML **inference** — the actual prediction at runtime, as opposed to training. These are called by the Node.js server via subprocess.

PREDICT_ALL.PY — PRIMARY RUNTIME PREDICTION

Called by: `runPythonPredictions()` and `runClusterAssignment()` in the server.

Model caching: Once loaded, models are cached at the Python module level, so the next call within the same process reuses the already-loaded artifacts. This is critical for the "What-If" simulator, which may call `predict_all.py` multiple times in succession with different parameter adjustments.

Prediction flow: (1) Load three risk artifacts (`model_survival.joblib` , `model_escalation.joblib` , `model_rv_dysfunction.joblib`) and the cluster model (`cluster_model.joblib`). (2) For each patient, engineer 185+ features matching the training pipeline — the `engineer_risk_features()` function exactly mirrors `ml_pipeline.py` 's `engineer_features()` . (3) For each target, impute missing values with the training cohort's median, standardize with the training cohort's z-score, and compute risk probability via `predict_proba()` . (4) For clustering, engineer 10 Zweck-style features via `engineer_cluster_features()` , impute, scale, optionally apply PCA (if PCA was the best clustering configuration), and compute Euclidean distance to each cluster centroid — the nearest centroid is the patient's assigned cluster, and inverse-distance-weighted similarities provide a confidence measure.

Output: A JSON object with `predictions[]` (patientId + survival/escalation/rv_dysfunction scores) and `clusters[]` (patientId + cluster_label, cluster_name, recommendation, distances, similarities).

PREDICT.PY — LEGACY SINGLE-BATCH SCRIPT

An earlier version of the prediction script, functionally superseded by `predict_all.py`. Still present in the codebase for backward compatibility. Loads models, engineers features, and returns risk scores for a single batch of patients. The newer script adds cluster assignment and model caching.

Runtime Integration

The server calls Python via `execFile`, which spawns a new Python process for each request. The communication protocol is simple: the server writes a JSON payload to the Python process's **stdin** and the Python process writes its JSON response to **stdout**. The server uses a list of candidate Python paths (starting with `PYTHON_PATH` env var, then several hardcoded system paths) and uses the first one that exists on disk. This handles the common problem of multiple Python installations on clinical workstations.

The Node.js `ml-random-forest` package runs an additional LOOCV (Leave-One-Out Cross-Validation) RandomForest on the server side for comparison — this is a legacy feature that provides a baseline survival probability using only 6 features (pre-RA, pre-PCWP, pre-CPO, pre-PAPI, pre-VIS, escalation status) rather than the full 185+ features the Python pipeline uses.

This section explains the machine learning methods used in this project in plain language — technical enough to be accurate, but accessible to a reader without formal ML training.

Random Forest

What it is: An ensemble of decision trees — typically hundreds of them. Each tree is trained on a random subset of the data (both rows and columns). The forest "votes" by averaging predictions across all trees.

Why it works: Individual decision trees overfit (they memorize noise in the training data), but averaging across many decorrelated trees cancels out the noise while preserving the signal. This makes Random Forests robust, resistant to overfitting, and capable of capturing non-linear relationships that linear models miss.

In this project: Used for survival and escalation prediction with 200 trees, max depth of 6 (limits how complex each tree can be — prevents overfitting on small data), and `class_weight="balanced"` (automatically gives more weight to the minority class — important since only ~25% of patients expired). The Random Forest is the best model for escalation (AUC 0.95) but barely beats random for survival (AUC 0.54).

Logistic Regression

What it is: A linear model that estimates the probability of a binary outcome (yes/no) using a weighted sum of input features, passed through a sigmoid function that squashes the output to a 0–1 range. Each feature gets a coefficient (positive = increases probability, negative = decreases).

Why it works: Simple, interpretable, and well-understood. The coefficients tell you exactly *how* each feature affects the outcome — an LVAD surgeon can understand what the model is doing. Logistic Regression also naturally produces well-calibrated probabilities (unlike Random Forests, which can be overconfident).

In this project: Best model for RV dysfunction (AUC 0.91). JSON coefficient files are exported so the model could theoretically be reimplemented in pure JavaScript without Python. The `class_weight="balanced"` parameter is critical — without it, a model trained on 75% survivors would learn to always predict "survive" and achieve 75% accuracy with zero clinical utility.

Consensus K-Means Clustering

What it is: An unsupervised learning method that groups similar patients together without being told what the "correct" groups are. K-Means works by placing K "centroids" in the data space and assigning each patient to the nearest centroid, then iteratively moving centroids to minimize the total distance.

Consensus extension: Standard K-Means is unstable — run it twice and you might get different clusters. Consensus K-Means runs K-Means 200 times on random 80% subsamples of the data, then measures *how consistently* each pair of patients co-clustered. This produces a consensus matrix (a heatmap where dark cells = patients who almost always cluster together) that is more stable than any single K-Means run.

Silhouette score: Measures how well-separated the clusters are, on a scale from -1 (terrible) to +1 (perfect). 0.237 means moderate separation — clusters are distinguishable but overlap somewhat. The clinical caution is important: cluster assignments should be treated as exploratory, not definitive diagnoses.

SHAP (SHapley Additive exPlanations)

What it is: A game-theoretic method for explaining model predictions. Imagine the model's features are "players" in a cooperative game, and the prediction is the "payout." SHAP computes each feature's fair contribution by averaging its marginal contribution across all possible feature subsets.

TreeSHAP: An efficient variant for tree-based models (Random Forest, XGBoost). Instead of retraining the model on all possible feature subsets, TreeSHAP exploits the tree structure to compute exact Shapley values in polynomial time.

Why it matters: SHAP values are consistent (a feature with higher true importance always gets a higher SHAP score), locally accurate (explanations match individual predictions), and the only method with a foundation in cooperative game theory. SHAP summary plots show which features matter most and whether high or low values increase risk — for example, SHAP might reveal that high lactate consistently increases mortality risk (red dots to the right of zero).

LASSO (L1-Regularized Logistic Regression)

What it is: Logistic Regression with an added penalty that is proportional to the sum of the absolute values of the coefficients. This penalty drives unimportant feature coefficients *exactly* to zero — performing automatic feature selection.

Why it matters: In a dataset with 185+ features but only ~100 patients, most features are noise.

LASSO automatically identifies the subset of features that actually matter, producing a simpler, more generalizable model. The regularization strength (controlled by the parameter C) was tuned via grid search across 30 values with 5-fold cross-validation to find the optimal sparsity level.

Cross-Validation (5-Fold Stratified)

What it is: A technique for evaluating model performance without a separate test set. The data is split into 5 equal "folds." The model is trained on 4 folds and tested on the remaining 1 fold, rotating 5 times so every patient is in the test set exactly once. "Stratified" means each fold preserves the same proportion of positive/negative outcomes as the full dataset.

Why it matters: Without cross-validation, you can easily overfit — a model might memorize the training data and achieve 99% accuracy on it but fail completely on new patients. 5-fold CV gives a realistic estimate of how the model will perform on unseen data.

ROC Curves and AUC

ROC Curve: Receiver Operating Characteristic curve — plots the True Positive Rate (sensitivity) against the False Positive Rate ($1 - \text{specificity}$) at every possible decision threshold. A perfect model goes straight up and then right (100% sensitivity at 0% false positives). A random model follows the diagonal.

AUC: The Area Under the ROC Curve — a single number summarizing model discrimination. 0.5 = random guessing, 0.7–0.8 = acceptable, 0.8–0.9 = excellent, > 0.9 = outstanding. AUC is *threshold-independent* — it measures how well the model separates positive from negative cases, regardless of where you set the cutoff.

In context: Escalation AUC = 0.95 means the model is outstanding — if you randomly pick one escalated patient and one non-escalated patient, the model will rank the escalated patient higher 95% of the time. Survival AUC = 0.54 means the model is barely better than a coin flip — you should not use it for decisions.

Feature Engineering

What it is: Transforming raw variables into features that better represent the underlying problem to the ML models. Raw data is rarely ML-ready — a single number like "lactate = 3.2" tells you less than "lactate went from 1.5 to 3.2 (rising trend)."

In this project: The most important engineered features are **delta features** (post – pre — captures the trajectory on Impella support) and **ratio features** (post / pre — captures the relative change). The mortality feature analysis confirmed that these trend features consistently outperform raw single-timepoint measurements. Other engineered features: BMI (from weight and

height), inotrope count (total vasoactive drugs — a proxy for shock severity), VIS-high indicator (boolean for vasopressor burden > 15), and SCAI numeric encoding (categorical staging → numeric values).

Class Imbalance and Missing Data

Class imbalance: When one outcome is much rarer than the other — in this cohort, ~25% mortality rate. Without correction, a model can achieve 75% accuracy by predicting "survive" for everyone, which is clinically useless. The project handles this with `class_weight="balanced"`, which automatically adjusts weights to give the minority class equal influence during training.

Missing data: Real clinical data is messy — echo and PV loop variables have 30–50% missingness. The pipeline drops columns that are 100% empty and imputes remaining missing values with the **median** of the training cohort. Median imputation is robust to outliers (unlike mean imputation) but may attenuate true clinical signals — this limitation is documented in every model report.

Beyond ML predictions, the platform provides five clinical decision support features designed to be immediately actionable at the bedside.

1. WEANING READINESS CHECKLIST

The weaning checklist evaluates 5 criteria that define whether a patient is ready to begin the Impella weaning process (gradually reducing pump flow to test native heart recovery):

CRITERION	THRESHOLD	WHY IT MATTERS
Cardiac Power Output (CPO)	≥ 0.60 W	The heart must generate adequate hydraulic power on its own. Below 0.6 W indicates severe myocardial depression.
Serum Lactate	< 2.0 mmol/L	Normal lactate means tissue oxygen delivery is adequate — no ongoing anaerobic metabolism from hypoperfusion.
Vasoactive-Inotropic Score (VIS)	< 10	Low vasopressor/inotrope requirement means the native heart is not dependent on pharmacologic support.
Pulmonary Artery Pulsatility Index (PAPI)	≥ 1.50	Good RV function is essential — Impella only supports the left ventricle, so the RV must function independently for weaning to succeed.
Extubation Status	Extubated (spontaneous breathing)	Spontaneous breathing reduces intrathoracic pressure swings, lowers RV afterload, and is a prerequisite for device explant.

The composite score is the percentage of criteria met (0–100%). A patient passes (`weaningPassed = true`) if they meet at least 4 of 5 criteria. The dashboard shows each criterion individually with its current value and pass/fail status, so the clinical team can see exactly *why* a patient is not yet a weaning candidate.

2. ESCALATION DANGER WARNINGS

Four clinical danger signals are monitored, plus the knowledge-base historical matching alert:

SIGNAL	THRESHOLD	CLINICAL MEANING
Right Atrial Pressure	> 20 mmHg	Severe RV congestion — the right heart cannot effectively pump blood forward, causing back-pressure into the venous system.
PAPI	< 1.00	Refractory RV shock — extremely low pulsatility indicates the RV is failing as a pump.
AST (Liver enzyme)	> 200 U/L	Hepatocellular damage from hepatic congestion (back-pressure from the failing RV) and/or hypoxic hepatitis (low cardiac output).
Lactate	> 3.0 mmol/L	Severe tissue perfusion deficit — sustained anaerobic metabolism despite Impella support.
Historical Ees/Ea Match	Within 15% tolerance	If the current Ees/Ea matches a historical patient who required escalation, the current patient gets flagged — pattern-based risk assessment without a model.

Any triggered signal raises a global `escalationWarning` flag. The dashboard shows a red alert badge, and individual danger signals are listed with their current values in the patient detail view.

3. EXPLAINABLE RISK DRIVERS

The risk drivers are the platform's most important interpretability feature. For each of the three outcomes (survival, escalation, RV dysfunction), the platform identifies up to 3 specific clinical measurements that are contributing to the patient's risk.

Each driver includes:

- **Feature** — the variable name (e.g., `postLactate`)
- **Impact** — a magnitude (positive = increases risk, negative = decreases risk)
- **Label** — a human-readable explanation (e.g., "Sustained tissue perfusion deficit (Lactate)")
- **Value** — the patient's current value for that metric

These drivers use **simple threshold rules**, not the opaque ML model — meaning they are transparent, clinically verifiable, and understandable. The thresholds are derived from published clinical guidelines and validated in the mortality feature importance analysis. This is intentional: a clinician should never have to trust a "black box." The ML model provides the overall risk probability; the rule-based drivers explain *why* the patient might be at risk using familiar clinical thresholds.

4. AI CLINICAL SUMMARY (GEMINI)

When configured with a `GEMINI_API_KEY` , the platform can generate a structured clinical handoff note using Google Gemini 1.5 Flash. The prompt includes all relevant patient data — demographics, pre/post hemodynamics, labs, device settings, risk scores, and weaning status — and instructs Gemini to produce a professional, highly-dense clinical memo. The model is explicitly instructed to use specific numbers, avoid speculative variables, and stay under 300 words with bullet-point formatting.

The fallback template-based summary is equally structured and provides the same information — it is written as a professional clinical memo with sections for clinical recommendation, RV state, perfusion and end-organ status, and actionable management pathway. The fallback runs when no API key is set or when Gemini's API call fails, ensuring the feature is always available.

5. "WHAT-IF" TREATMENT SIMULATOR

The simulator allows clinicians to test "what happens if we adjust the flow by 0.5 L/min or lower the lactate with aggressive diuresis?" It accepts adjustments to Impella flow, performance level, VIS, lactate, RA, PAPI, and CPO — then recalculates risk scores, weaning status, escalation warnings, and risk drivers dynamically. The simulator re-runs the full ML prediction pipeline on the modified patient data, so the new risk scores reflect the full 185+ feature model, not a simple heuristic. This is a powerful tool for treatment planning — a clinician can test multiple weaning scenarios before actually reducing pump support at the bedside.

This section summarizes the most important findings from the ML training, feature importance analysis, clustering, and effectiveness analysis.

Model Performance Summary

MODEL	TARGET	BEST ALGORITHM	CV AUC	CLINICAL UTILITY
Survival	Hospital mortality	RandomForest	0.54	Not clinically useful
MCS Escalation	ECMO / LVAD / transplant / arrest	RandomForest	0.86–0.95	Clinically useful
RV Dysfunction	Composite RV failure criteria	LogisticRegression	0.91	Clinically useful
PV Loop Escalation	Escalation from PV loop features only	LogisticRegression	~0.85	Clinically useful (specialized)

Top Mortality Predictors (Consensus Across 5 Methods)

RANK	FEATURE	CATEGORY	CONSENSUS SCORE	CLINICAL SIGNIFICANCE
1	post_ast	Liver function	66.0%	Post-implant AST — hepatocyte injury from low cardiac output. Ranked #1 by 3/5 methods.
2	post_lactate	Tissue perfusion	43.2%	Direct marker of tissue hypoperfusion and anaerobic metabolism.
3	age	Demographics	43.0%	Chronological age — robust univariate predictor across all methods.
4	post_hco3	Acid-base	42.0%	Bicarbonate reflects metabolic compensation in shock state.

RANK	FEATURE	CATEGORY	CONSENSUS SCORE	CLINICAL SIGNIFICANCE
5	post_egfr	Renal function	39.5%	Post-implant eGFR — AKI in cardiogenic shock portends poor outcomes.
6	post_alt	Liver function	39.4%	Parallels AST as hepatocyte injury marker.
7	post_map	Hemodynamics	39.0%	Mean arterial pressure — adequacy of end-organ perfusion.
8	vis_score	Vasopressor burden	38.9%	Vasoactive-Inotropic Score quantifies total vasopressor support.
9	pre_hco3	Acid-base (baseline)	37.6%	Pre-implant bicarbonate — baseline metabolic reserve.
10	ratio_ra	RV function trend	34.3%	Ratio of post/pre right atrial pressure — RV failure trajectory.

Key insight: Hepatocyte injury (AST/ALT) dominates the mortality signal. Rising AST/ALT on Impella support may be the **earliest warning signal** of impending mortality — before traditional hemodynamic thresholds (RA, PAPI, CPO) cross their danger zones. This is clinically plausible: the liver is highly sensitive to both low forward flow (hypoxic hepatitis) and backward congestion (hepatic congestion from RV failure), making it an early indicator of both problems.

Feature Reduction Analysis

TARGET	FULL MODEL AUC	BEST SUBSET AUC	DELTA	INTERPRETATION
Survival	0.59	0.62 (top-10 features)	+0.03	Top-10 consensus features match full set — remaining features add noise
Escalation	0.94	0.84 (top-50 features)	−0.10	Full feature set needed — escalation is a multi-system phenomenon

TARGET	FULL MODEL AUC	BEST SUBSET AUC	DELTA	INTERPRETATION
RV Dysfunction	0.93	0.80 (top-50 features)	−0.13	Full feature set needed — RV dysfunction involves complex interactions

Recommendation: Use a simplified 10-feature model for mortality screening, but maintain the full model for escalation and RV dysfunction prediction.

Cluster Phenotype Summary

CLUSTER	N	SURVIVAL	ESCALATION	RENAL FAILURE	KEY SIGNATURE
Non-congested (Low-risk)	27	70.4%	14.8%	7.4%	Younger (51), higher BMI (31.5), preserved eGFR (79.7), lactate 2.12, SCAI ~C
Cardiorenal (Moderate-risk)	37	67.6%	20.0%	22.9%	Older (65), lower BMI (25.7), low eGFR (41.0), lactate 1.70, SCAI ~C
Cardiometabolic (High-risk)	4	50.0%	0.0%	25.0%	Extreme ALT (1331), leukocytosis (WBC 18.0), very high lactate (3.38), SCAI ~D

Important caveat: Survival differences between clusters are small (70.4% vs 67.6% — only 2.8 percentage points between Non-congested and Cardiorenal). The "Non-congested" cluster paradoxically has elevated lactate compared to "Cardiorenal" — a **perfusion-stress signature** that clustering identifies as a natural grouping but which does not correspond to better outcomes in the expected direction. This underscores that clustering discovers empirical groups, not guaranteed clinical truths.

Clinical Recommendations

1. **Bedside mortality screening:** Monitor AST, lactate, HCO_3 , eGFR, and age. A composite of these 5 features provides equivalent information to the full 185-feature model for survival prediction.
2. **Escalation planning:** Maintain the full feature model for escalation risk. The rich feature set captures the multi-organ dysfunction pattern that precedes clinical decompensation.
3. **RV dysfunction monitoring:** Prioritize post-implant PAPI, RA, and TAPSE alongside the delta features of PVR and PCWP. RV-PA uncoupling (rising PVR with falling PAPI) is the dominant RV failure pathway.
4. **Data completeness:** Post-implant labs (AST, lactate, HCO_3 , eGFR) provide more signal than pre-implant values. Prioritize collecting 48-hour post-implant labs over expanding the pre-implant panel.
5. **Use both models:** The escalation and RV dysfunction models are strong enough to support clinical decision-making. The survival model is not — treat mortality risk scores as research-grade information only.

Limitations & Future Directions

Every analytical tool has limitations. This section documents what we know the platform cannot do, where the data is weak, and what directions would strengthen future versions.

DATA LIMITATIONS

- **Sample size:** 112–128 patients from a single institution. This is a small dataset for ML — feature importance rankings, cluster boundaries, and AUC estimates may all shift with larger cohorts. Models trained on 100 patients typically lose 5–15% AUC when applied to a new hospital's population.
- **Missing data:** Echo and PV loop variables have 30–50% missingness. Median imputation may attenuate true clinical signals. The RV dysfunction target relies on echo data missing for ~40% of patients, potentially introducing bias.
- **Class imbalance:** ~25% mortality rate. While handled via `class_weight` balancing, rare-but-important signals may still be missed.
- **Associational, not causal:** All findings are statistical associations. Confounding by indication (sicker patients get more aggressive treatment) and unmeasured variables cannot be excluded.
- **Single-center:** All data comes from one institution's Impella program. Practice patterns, patient demographics, and referral patterns may differ significantly at other centers.

MODEL LIMITATIONS

- **Survival model is unreliable:** AUC ~0.54 is barely above random chance. Survival in this cohort is likely driven by factors not captured in pre/post implant RHC data — refractory arrhythmias, multi-organ failure trajectory, withdrawal of care decisions, or complications unrelated to hemodynamics. Do **not** use the mortality risk score for clinical decision-making until validated on a larger cohort.
- **No external validation:** All models have only been internally cross-validated. External validation on an independent cohort from a different institution is the single most important next step before any clinical deployment.
- **Cluster stability:** Silhouette score of 0.237 indicates moderate separation at best. Clusters overlap, and assignments can shift with new data. The cardiometabolic cluster has only 4 patients — too few for reliable profiling.
- **Ees/Ea matching:** The knowledge base pattern-matching alert uses a 15% tolerance on Ees/Ea. This is a heuristic, not a statistically validated threshold.

FUTURE DIRECTIONS

1. **Multi-center validation:** The highest priority. Models should be tested on data from at least 2–3 other institutions before any clinical deployment.
2. **Real-time inference:** Currently batch-based (upload file → process → visualize). Real-time streaming from electronic health records (EHR) would allow continuous risk score updates as new labs and vitals become available.
3. **Expanded feature space:** Continuous hemodynamic waveforms (arterial line, pulmonary artery catheter) could provide richer features than discrete timepoints. PV loop features from real-time pressure-volume catheters would strengthen the V-A coupling analysis.
4. **Longitudinal tracking:** Current analysis is pre/post (two timepoints). A time-series model tracking daily hemodynamics over the full support period would capture dynamic trajectories more accurately.
5. **Survival model improvement:** Integrating non-hemodynamic variables (comorbidity indices, frailty scores, code status, laboratory trajectory velocities) may improve survival prediction beyond the current near-random AUC.
6. **Threshold optimization:** The current 0.5 decision threshold is arbitrary. Cost-sensitive threshold tuning — where false negatives (missing a high-risk patient) are weighted 5–10× more than false positives — would better match clinical reality.
7. **Explainable AI dashboard:** SHAP waterfall plots per patient (showing exactly which features drive the individual prediction) would enhance the risk driver feature. Currently, the risk drivers use rule-based thresholds rather than the actual ML model's SHAP values.
8. **Patient-reported outcomes:** Quality of life, functional capacity (6-minute walk test), and heart failure symptoms (NYHA class) after discharge would provide a more complete picture of recovery beyond survival.

Bottom line: This platform is a **research-grade decision support tool**. The escalation and RV dysfunction models are strong enough to inform clinical thinking, but no model output should replace clinical judgment. The survival model should not be used for triage. External validation is essential before any clinical deployment. The platform's greatest value may be as an **educational and**

exploratory tool — helping clinicians understand the multivariate drivers of outcomes in their Impella population, identify data quality gaps, and generate hypotheses for prospective studies.

Impella Analytics — Project Documentation
Generated May 23, 2026 · For clinical research use only

React 19 + Express 4 + Vite 6 + Python 3 · scikit-learn · SHAP · weasyprint